
Guía de referencia · Data Science & ML

Preprocesamiento y Calidad del Dato en Machine Learning

Preprocesamiento · Valores Faltantes · Outliers · Leakage
Escalado · Codificación · Reducción · NLP · Desbalanceo · Pipelines

v1.0 · 2025

Índice

1. Data Quality

- 1.1. ¿Qué es Data Quality?
- 1.2. Dimensiones de Data Quality
- 1.3. Tipos de Checks de Data Quality
- 1.4. Métricas Clave de Data Quality

2. Integración de Datos

- 2.1. Integración y Fusión del Dato
- 2.2. Buenas Prácticas en Integración del Dato

3. Pipelines

- 3.1. Puesta en Producción y Pipelines
 - Pipeline completo con scikit-learn
 - Monitorización en producción
 - Buenas prácticas de producción

4. Valores Faltantes (NAs)

- 4.1. Valores Faltantes: Tipos y Diagnóstico
 - Tipos de valores faltantes
 - Diagnóstico en Python
- 4.2. Tratamiento de Valores Faltantes
 - Estrategia según situación

5. Valores Atípicos (Outliers)

- 5.1. Concepto y Tipos
- 5.2. Detección de Valores Atípicos
 - Métodos univariantes
 - Métodos bivariantes
 - Métodos multivariantes
- 5.3. Tratamiento de Valores Atípicos
 - Estrategias disponibles
 - Decisión según el tipo de problema
 - Sensibilidad de modelos a outliers

6. Filtración de Datos (Data Leakage)

- 6.1. Data Leakage
 - Tipos de leakage
 - Cómo evitarlo
 - Orden correcto del pipeline

7. Separación, Escalado, Normalización y Codificación

- 7.1. Split del Conjunto de Datos
- 7.2. Escalado de Datos
- 7.3. Normalización de Datos

7.4. Codificación de Variables Categóricas

- Tipos de variables categóricas
- Métodos de codificación

8. Ingeniería de Variables Temporales (Cómo tratar Fechas)

8.1. Ingeniería de Variables Temporales

- Extracción de componentes básicos
- Variables cíclicas (seno/coseno)
- Transformaciones avanzadas para series temporales

9. Reducción de Dimensionalidad

9.1. Reducción de la Dimensionalidad

- Feature Selection (Selección de variables)
- Feature Extraction (Extracción de variables)

10. Procesamiento y Representación de Texto (NLP)

10.1. Procesamiento de Texto (NLP)

- Pipeline típico de NLP
- Limpieza de texto
- Tokenización
- Stopwords
- Normalización: Stemming vs. Lematización

10.2. Representaciones de Texto

- Bag of Words (BoW)
- TF-IDF
- Embeddings

11. Desbalanceo de Datos

11.1. Desbalanceo de Datos: Concepto y Métricas

- Detección del desbalanceo
- Métricas adecuadas para datos desbalanceados
- ¿Qué métrica elegir?

11.2. Técnicas para Conjuntos Desbalanceados

- A nivel de datos
- A nivel de algoritmo

1. Data Quality

1.1. ¿Qué es Data Quality?

Los modelos de Machine Learning **dependen completamente de los datos de entrada**. Datos erróneos producen modelos imprecisos — principio conocido como "**garbage in, garbage out**". Una parte muy significativa del tiempo en cualquier proyecto de Data Science se dedica a la **limpieza y preparación de datos**.

Data Quality mide el grado en que los datos son adecuados para el uso que se les quiere dar. Su evaluación **depende siempre del contexto** del problema.

1.2. Dimensiones de Data Quality

Antes de trabajar con cualquier dataset, conviene revisar estas **seis dimensiones**:

Dimensión	Pregunta clave	Ejemplo de problema
Exactitud	¿El dato representa la realidad?	Edad negativa, salario imposible
Completitud	¿Faltan valores?	Columnas con muchos NaN
Consistencia	¿Se contradicen los datos?	País=Alemania, Ciudad=Roma
Validez	¿Los datos cumplen el formato esperado?	Email malformado, fecha inválida
Actualidad	¿Los datos están actualizados?	Precios de vivienda de hace 50 años
Unicidad	¿Hay registros duplicados?	El mismo cliente aparece dos veces

1.3. Tipos de Checks de Data Quality

Existen cuatro tipos de validaciones que se suelen implementar:

- **Reglas de negocio**: validaciones basadas en la lógica del dominio. *Ejemplo: la fecha de envío debe ser posterior a la de pedido.*
- **Validaciones estadísticas**: detección de outliers, cambios en distribuciones, picos anómalos.
- **Restricciones**: integridad estructural del dataset (claves primarias, campos no nulos, unicidad).
- **Validaciones de esquema**: los tipos de datos, nombres de columnas y campos obligatorios son los esperados.

1.4. Métricas Clave de Data Quality

Las más habituales para monitorizar la calidad son:

- **% de valores faltantes** por columna — columnas con **más del 20-30%** requieren análisis adicional.
- **Número de duplicados** — especialmente en campos identificadores (`order_id`, `client_id...`).
- **Distribuciones anómalas** — picos inusuales, cambios bruscos en medias respecto a históricos.
- **Violaciones de reglas** — registros que incumplen restricciones definidas (fechas inválidas, importes negativos...).

2. Integración de Datos

2.1. Integración y Fusión del Dato

En la mayoría de proyectos los datos **proviene de múltiples fuentes**: bases de datos internas, APIs, data lakes, archivos externos. Al combinarlos surgen problemas habituales como **formatos diferentes, claves inconsistentes, duplicados o registros incompletos**.

Operaciones de integración en Python (pandas)

```
# Merge (equivalente a SQL JOIN) – une por columna clave
df_merged = pd.merge(df1, df2, on='id_cliente', how='left')
# how: 'inner', 'left', 'right', 'outer'

# Concat – apila filas o columnas
df_total = pd.concat([df1, df2], axis=0, ignore_index=True)
```

Tipos de JOIN

Tipo	Resultado
INNER JOIN	Solo registros que coinciden en ambas tablas
LEFT JOIN	Todos los de la tabla izquierda + coincidencias de la derecha
RIGHT JOIN	Todos los de la tabla derecha + coincidencias de la izquierda
FULL OUTER JOIN	Todos los registros de ambas tablas

2.2. Buenas Prácticas en Integración del Dato

Validaciones tras el join

Después de integrar, **siempre revisar** que el proceso no haya comprometido la calidad:

```
# Número de registros antes y después
print(f"Registros antes: {len(df1)}")
print(f"Registros después del join: {len(df_merged)}")

# % de registros emparejados (en LEFT JOIN)
emparejados = df_merged['columna_de_df2'].notna().sum()
print(f"% emparejados: {emparejados / len(df_merged) * 100:.1f}%")

# Duplicados tras el join
print(f"Duplicados: {df_merged.duplicated().sum()}")

# Claves únicas
assert df_merged['id_cliente'].is_unique, "¡Hay claves duplicadas!"
```

Pipeline recomendado para integración

1. **Limpiar** cada dataset individualmente antes de integrar.

2. **Estandarizar formatos** (tipos de datos, nombres de columnas, encoding).
3. **Integrar** los datos.
4. **Validar consistencia** del resultado (registros, duplicados, nulos, claves).
5. **Análisis exploratorio** del dataset integrado.

3. Pipelines

3.1. Puesta en Producción y Pipelines

Un **pipeline** encadena todos los pasos de preprocesamiento y modelado en una secuencia reproducible que garantiza que las mismas transformaciones se apliquen tanto en entrenamiento como en inferencia, sin riesgo de leakage.

Pipeline completo con scikit-learn

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier

num_features = ['edad', 'ingresos', 'antiguedad']
cat_features = ['pais', 'tipo_cliente']

num_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', num_transformer, num_features),
    ('cat', cat_transformer, cat_features)
])

pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', RandomForestClassifier(class_weight='balanced', random_state=42))
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

import joblib
joblib.dump(pipeline, 'pipeline_modelo.pkl')

pipeline_cargado = joblib.load('pipeline_modelo.pkl')
predicciones = pipeline_cargado.predict(nuevos_datos)
```

Monitorización en producción

Una vez desplegado el modelo, hay que vigilar que su rendimiento no se degrade con el tiempo:

Fenómeno	Descripción	Señal de alerta
Data Drift	La distribución de los datos de entrada cambia	Las variables tienen medias o varianzas distintas a las del entrenamiento
Model Drift	El rendimiento del modelo se degrada	Las métricas de negocio empeoran progresivamente

Buenas prácticas de producción

- **Versionado de datos:** registrar qué versión del dataset se usó para entrenar cada modelo.
- **Versionado del pipeline:** guardar el pipeline serializado junto con sus parámetros.
- **Logging del preprocesado:** registrar cambios, errores y decisiones de transformación.
- **Mismo pipeline en entrenamiento e inferencia:** nunca re-ajustar transformadores con datos de producción.

4. Valores Faltantes (NAs)

4.1. Valores Faltantes: Tipos y Diagnóstico

Un **valor faltante** es cualquier dato no registrado. En Python se representa habitualmente como NaN, None, Null o NA.

Pueden aparecer por **errores de captura, sensores defectuosos, usuarios que no responden o campos no aplicables** en ciertos contextos.

Impacto: muchos modelos de ML no aceptan valores faltantes, reducen el tamaño efectivo del dataset y pueden **introducir sesgo**.

Tipos de valores faltantes

Tipo	Descripción	Ejemplo
MCAR (Missing Completely At Random)	Aleatorios, sin relación con ninguna variable	Fallo puntual del sistema en un registro
MAR (Missing At Random)	Dependen de variables observadas, no del valor faltante	Tensión no tomada porque ya hubo revisión reciente
MNAR (Missing Not At Random)	La ausencia está relacionada con el propio valor no registrado	Usuario no reporta actividad física baja para evitar prejuicios

Diagnóstico en Python

```
import pandas as pd
import missingno as msno

# % de valores faltantes por columna
df.isnull().sum() / len(df) * 100

# Visualización de patrones de valores faltantes
msno.matrix(df)
msno.heatmap(df) # Correlación entre variables con faltantes
```

4.2. Tratamiento de Valores Faltantes

Estrategia según situación

Situación	Método recomendado
MCAR , bajo % de faltantes	Imputación simple (media, mediana o moda)
MCAR , muy pocos casos	Eliminar los registros
MAR (dependen de otras variables)	Imputación avanzada (KNN, Regresión)

Situación	Método recomendado
MNAR (dependen del propio valor)	Indicador de faltante (missing flag) + imputación
Variable con muchos faltantes	Eliminar la variable o reconsiderar su uso
Variable importante para el modelo	Imputación avanzada para preservar información
Categorica con pocos faltantes	Imputar por moda o crear categoría "desconocido"

```

# Eliminación de filas con faltantes
df.dropna(inplace=True)

# Eliminación de columnas con muchos faltantes (ej: >50%)
df.drop(columns=df.columns[df.isnull().mean() > 0.5], inplace=True)

# Imputación simple (numérica)
df['edad'].fillna(df['edad'].median(), inplace=True)

# Imputación simple (categórica)
df['categoria'].fillna(df['categoria'].mode()[0], inplace=True)

# Imputación avanzada con KNN
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=5)
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

# Missing flag – cuando la ausencia es información en sí misma
df['ingresos_missing'] = df['ingresos'].isnull().astype(int)

```

5. Valores Atípicos (Outliers)

5.1. Concepto y Tipos

Un **valor atípico** (outlier) es una observación que se aleja significativamente del patrón general de los datos. Estadísticamente, aparece en las **colas de la distribución**, muy alejado de la media o mediana. Su interpretación **depende siempre del contexto**: puede ser un error de captura, un evento raro válido, o una observación con alto valor informativo (fraude, cliente VIP, sensor defectuoso...).

Valor atípico $\neq$ Anomalía: un outlier es un valor estadísticamente extremo; una anomalía es un comportamiento inesperado respecto al sistema. No siempre coinciden.

5.2. Detección de Valores Atípicos

Métodos univariantes

Se analiza cada variable de forma individual. Son simples y rápidos.

- **Z-score:** mide cuántas desviaciones estándar se aleja un valor de la media. Valores con $|z| > 3$ suelen considerarse atípicos.
- **IQR (Rango Intercuartílico):** considera atípico todo valor que se aleje más de **1.5 x IQR** por encima del percentil 75 o por debajo del percentil 25.
- **Boxplot:** visualización que muestra extremos, asimetría e IQR de forma simultánea.

```
import numpy as np
from scipy import stats

# Z-score
z_scores = np.abs(stats.zscore(df['columna']))
outliers_z = df[z_scores > 3]

# IQR
Q1 = df['columna'].quantile(0.25)
Q3 = df['columna'].quantile(0.75)
IQR = Q3 - Q1
outliers_iqr = df[(df['columna'] < Q1 - 1.5*IQR) | (df['columna'] > Q3 + 1.5*IQR)]

# Boxplot visual
import matplotlib.pyplot as plt
df['columna'].plot(kind='box')
```

Métodos bivariantes

Se analiza la relación entre **dos variables**. Un valor puede ser normal en cada variable individualmente pero atípico en su combinación (ej: persona de 20 años con ingresos de 5M€). Se detecta con **scatter plots**.

```
import seaborn as sns
sns.scatterplot(data=df, x='edad', y='ingresos')
```

Métodos multivariantes

Un valor puede no ser atípico en ninguna variable individual pero sí en la **combinación de varias**. Requieren métodos más avanzados.

- **Distancia de Mahalanobis**: mide la distancia al centro de la distribución considerando correlaciones entre variables.
- **Isolation Forest**: algoritmo basado en árboles, muy eficiente para datasets grandes.
- **Métodos basados en densidad**: DBSCAN, Local Outlier Factor (LOF).

```
from sklearn.ensemble import IsolationForest

model = IsolationForest(contamination=0.05, random_state=42)
df['outlier'] = model.fit_predict(df[['col1', 'col2', 'col3']])
# -1 = outlier, 1 = normal
```

5.3. Tratamiento de Valores Atípicos

Antes de actuar, pregúntate siempre: ¿Es un error? ¿Es un evento raro válido? ¿Es relevante para el negocio? **Eliminar sin analizar puede suponer perder información valiosa.**

Estrategias disponibles

Eliminación — útil cuando son errores evidentes. Reduce ruido, pero con riesgo de perder información.

```
df = df[df['columna'] < umbral_maximo]
```

Transformación — reduce la influencia de extremos sin eliminar datos.

```
import numpy as np
df['columna_log'] = np.log1p(df['columna']) # Log
df['columna_sqrt'] = np.sqrt(df['columna']) # Raíz cuadrada
```

Winsorización — limita los valores a ciertos percentiles. Muy usada en **datos financieros**.

```
from scipy.stats.mstats import winsorize
df['columna_wins'] = winsorize(df['columna'], limits=[0.01, 0.01])
```

Imputación — sustituye extremos por estimaciones. Usar con precaución.

Modelos robustos — a veces la mejor opción es no tocar el dato y elegir un modelo tolerante.

Decisión según el tipo de problema

Tipo de problema	Estrategia recomendada
Predicción de demanda	Suavizar (transformación)
Detección de fraude	Mantener outliers
Errores de sensores	Eliminar
Datos financieros	Winsorización

Sensibilidad de modelos a outliers

Modelo	Sensibilidad a outliers
K-means	Muy alta
Regresión lineal	Alta
KNN	Alta
Árboles de decisión / Random Forest	Baja

6. Filtración de Datos (Data Leakage)

6.1. Data Leakage

El **data leakage** (fuga de datos) ocurre cuando el modelo utiliza información que **no estaría disponible en el momento real de la predicción**. Introduce un sesgo optimista: el modelo parece funcionar mejor de lo que realmente lo hará en producción.

Tipos de leakage

Tipo	Descripción	Ejemplo
Temporal	Se usan datos del futuro para predecir el pasado	Entrenar con transacciones posteriores a la fecha de predicción
Por target	Se incluyen variables directamente relacionadas con la variable objetivo	Incluir el importe de una devolución para predecir si habrá devolución
En preprocesamiento	Se aplican transformaciones usando datos de test/validación	Calcular la media para imputar usando todo el dataset, no solo el de entrenamiento

El leakage en preprocesamiento es el **más frecuente y silencioso**: escalar o imputar con estadísticas calculadas sobre el dataset completo contamina el modelo con información del test.

Cómo evitarlo

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# 1. Separar PRIMERO, transformar DESPUÉS
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Ajustar el scaler SOLO sobre train, aplicar a ambos
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit + transform
X_test_scaled = scaler.transform(X_test)      # solo transform, nunca fit

# 3. Usar Pipeline para automatizarlo y evitar errores manuales
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', SomeModel())
])
pipe.fit(X_train, y_train) # el scaler solo ve X_train internamente
```

Orden correcto del pipeline

1. **Split** del conjunto de datos (train / validación / test).
2. **Ajustar** las transformaciones con datos de entrenamiento (fit).

3. **Aplicar** las transformaciones al test (transform).
4. **Entrenar** el modelo.

7. Separación, Escalado, Normalización y Codificación

7.1. Split del Conjunto de Datos

El dataset se divide en **tres subconjuntos** con roles distintos:

Subconjunto	Uso
Entrenamiento	El modelo aprende patrones y relaciones
Validación	Se ajustan hiperparámetros y se comparan modelos; evita sobreajuste
Test	Evaluación final del rendimiento real; el modelo nunca lo ha visto

```
from sklearn.model_selection import train_test_split

# Split en dos pasos: primero separar test, luego validación del resto
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42)

# Resultado: 60% train / 20% val / 20% test
```

En **datos temporales**, nunca se hace un split aleatorio: el test siempre corresponde a los datos más recientes para respetar el orden cronológico.

7.2. Escalado de Datos

El escalado transforma las variables numéricas para que tengan **una escala común**. Sin escalado, variables con magnitudes grandes dominan artificialmente sobre variables con magnitudes pequeñas.

Modelos basados en distancia o gradiente (KNN, SVM, regresión logística, redes neuronales) son muy sensibles a la escala. Árboles de decisión y Random Forest son robustos y **no requieren escalado**.

Método	Cómo funciona	Sensibilidad a outliers	Cuándo usarlo
Min-Max Scaler	Escala a rango [0, 1]	Alta	Datos sin outliers, cuando se necesita rango fijo
Standard Scaler	Media = 0, desviación estándar = 1	Media	Datos normales, modelos basados en distancia
Robust Scaler	Usa mediana e IQR (percentiles 25-75)	Baja	Datos con muchos valores atípicos

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler

scaler = MinMaxScaler()    # rango [0, 1]
scaler = StandardScaler()  # media 0, std 1
scaler = RobustScaler()    # resistente a outliers

# Siempre: fit en train, solo transform en test
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

7.3. Normalización de Datos

La normalización es distinta al escalado: mientras el escalado actúa **por columna** (variable a variable), la normalización actúa **por fila** (observación a observación).

Regla nemotécnica: escalado = comparar variables entre sí. Normalización = comparar observaciones entre sí.

Norma	Cómo funciona	Resultado	Cuándo usar
L1	Divide por la suma de valores absolutos	Suma del vector = 1	Mantener proporciones y dispersión
L2	Divide por la norma euclídea	Norma del vector = 1	Modelos basados en distancia (KNN, SVM)
L∞ / Max	Divide por el valor máximo absoluto	Valor máximo = 1	Método simple, sin outliers

```
from sklearn.preprocessing import Normalizer

normalizer = Normalizer(norm='l2')  # opciones: 'l1', 'l2', 'max'
X_normalized = normalizer.fit_transform(X)
```

7.4. Codificación de Variables Categóricas

Los modelos de ML trabajan con **valores numéricos**, por lo que las variables categóricas deben transformarse. Una mala codificación puede introducir relaciones artificiales o sesgar el modelo.

Tipos de variables categóricas

- **Nominales:** sin orden natural entre categorías (colores, países, tipos de producto).
- **Ordinales:** con jerarquía definida pero sin distancia numérica exacta (nivel educativo, satisfacción: baja/media/alta).

Métodos de codificación

Método	Cómo funciona	Ventajas	Inconvenientes	Cuándo usar
One-Hot	Una columna binaria por categoría	No introduce orden artificial	Alta dimensionalidad	Variables nominales con pocas categorías
Label Encoding	Asigna un entero a cada categoría	Simple, eficiente	Introduce orden artificial	Variables ordinales
Target Encoding	Sustituye por la media del target	Reduce dimensionalidad	Riesgo de leakage	Muchas categorías (con cuidado)
Hashing	Función hash a columnas fijas	No necesita conocer categorías	Colisiones, menos interpretable	Alta cardinalidad
Embeddings	Representación vectorial aprendida	Captura relaciones complejas	Requiere Deep Learning	Alta cardinalidad, DL

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder
from category_encoders import TargetEncoder

# One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=['pais', 'tipo_producto'], drop_first=True)

# Label Encoding (solo para variables ordinales)
le = LabelEncoder()
df['nivel_edu'] = le.fit_transform(df['nivel_edu'])

# Ordinal Encoding (orden explícito)
oe = OrdinalEncoder(categories=[['bajo', 'medio', 'alto']])
df[['satisfaccion']] = oe.fit_transform(df[['satisfaccion']])

# Target Encoding (ajustar SÓLO con datos de entrenamiento)
te = TargetEncoder()
df_train['ciudad'] = te.fit_transform(df_train['ciudad'], y_train)
df_test['ciudad'] = te.transform(df_test['ciudad'])
```

El **Target Encoding** aplicado sobre todo el dataset (incluyendo test) provoca leakage. Siempre dentro de un pipeline o ajustado únicamente con datos de entrenamiento.

8. Ingeniería de Variables Temporales (Cómo tratar Fechas)

8.1. Ingeniería de Variables Temporales

Las **fechas brutas** no aportan valor directamente a un modelo: hay que descomponerlas en variables que capturen **tendencias, estacionalidad y patrones cíclicos**.

Extracción de componentes básicos

```
df['fecha'] = pd.to_datetime(df['fecha'])

df['anyo']      = df['fecha'].dt.year
df['mes']       = df['fecha'].dt.month
df['dia']       = df['fecha'].dt.day
df['dia_semana'] = df['fecha'].dt.dayofweek # 0=lunes, 6=domingo
df['es_fin_semana'] = df['dia_semana'].isin([5, 6]).astype(int)
df['trimestre'] = df['fecha'].dt.quarter
```

Variables cíclicas (seno/coseno)

Algunas variables son cíclicas: diciembre está "cerca" de enero, y las 23:00 están cerca de las 0:00. Un valor numérico simple no captura esa circularidad. La solución es codificar con **seno y coseno**:

```
import numpy as np

# Mes como variable cíclica (rango 1-12)
df['mes_sin'] = np.sin(2 * np.pi * df['mes'] / 12)
df['mes_cos'] = np.cos(2 * np.pi * df['mes'] / 12)

# Hora del día (rango 0-23)
df['hora_sin'] = np.sin(2 * np.pi * df['hora'] / 24)
df['hora_cos'] = np.cos(2 * np.pi * df['hora'] / 24)
```

Transformaciones avanzadas para series temporales

```
# Lags (retardos): valores pasados como variables predictoras
df['ventas_lag_1'] = df['ventas'].shift(1) # ventas de ayer
df['ventas_lag_7'] = df['ventas'].shift(7) # ventas de hace 7 días

# Rolling windows (ventanas móviles): suavizan la serie y capturan tendencias
df['media_7d'] = df['ventas'].rolling(window=7).mean()
df['max_30d'] = df['ventas'].rolling(window=30).max()

# Deltas (diferencias): capturan el cambio entre periodos consecutivos
df['delta_1d'] = df['ventas'].diff(1) # variación respecto al día anterior
```

Leakage temporal: en series temporales el split debe respetar el orden cronológico. Entrenamiento = pasado, test = futuro. **Nunca** usar `train_test_split` aleatorio con datos temporales.

```
# Split temporal correcto
split_idx = int(len(df) * 0.8)
train = df.iloc[:split_idx]
test  = df.iloc[split_idx:]
```

9. Reducción de Dimensionalidad

9.1. Reducción de la Dimensionalidad

A medida que aumenta el número de variables, el modelo se vuelve más complejo y propenso al sobreajuste. Este fenómeno se conoce como **maldición de la dimensionalidad**.

Objetivos de la reducción: eliminar variables irrelevantes o redundantes, reducir el riesgo de sobreajuste, disminuir el coste computacional y, en algunos casos, mejorar la interpretabilidad.

Feature Selection (Selección de variables)

Elimina variables manteniendo las originales — **preserva la interpretabilidad**.

Tipo	Descripción	Ejemplos
Filtro	Evalúa cada variable de forma independiente mediante métricas estadísticas	Correlación, chi-cuadrado, varianza
Wrapper	Evalúa combinaciones de variables entrenando un modelo	RFE (Recursive Feature Elimination)
Embedded	La selección se realiza dentro del propio algoritmo	Lasso, importancia en Random Forest

```
from sklearn.feature_selection import SelectKBest, f_classif, RFE
from sklearn.ensemble import RandomForestClassifier

# Filtro: seleccionar las K mejores variables
selector = SelectKBest(score_func=f_classif, k=10)
X_new = selector.fit_transform(X_train, y_train)

# Wrapper: RFE con un modelo base
rfe = RFE(estimator=RandomForestClassifier(), n_features_to_select=10)
X_rfe = rfe.fit_transform(X_train, y_train)

# Embedded: importancia de variables del Random Forest
model = RandomForestClassifier().fit(X_train, y_train)
importances = pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False)
```

Feature Extraction (Extracción de variables)

Combina variables creando nuevas representaciones — **puede perder interpretabilidad**.

Técnica	Descripción	Cuándo usar
PCA	Crea combinaciones lineales que maximizan la varianza explicada	Datos numéricos correlacionados, sin necesidad de interpretabilidad

Técnica	Descripción	Cuándo usar
LDA	Busca combinaciones que separen mejor las clases (supervisado)	Clasificación con clases bien definidas
Autoencoders	Redes neuronales que aprenden una representación comprimida	Datos complejos, relaciones no lineales

```

from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# PCA: reducir a N componentes principales
pca = PCA(n_components=10)
X_pca = pca.fit_transform(X_train)

# Ver cuánta varianza acumulada explican los componentes
print(pca.explained_variance_ratio_.cumsum())

# LDA: supervisado, reduce a n_clases - 1 componentes maximo
lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X_train, y_train)

```

10. Procesamiento y Representación de Texto (NLP)

10.1. Procesamiento de Texto (NLP)

El texto es un dato **no estructurado** que no puede alimentarse directamente a un modelo de ML. Antes de representarlo numéricamente, hay que limpiar, tokenizar y normalizar.

Pipeline típico de NLP

Texto bruto -> Limpieza -> Tokenización -> Stopwords -> Normalización -> Representación

Limpieza de texto

```
import re

def limpiar_texto(texto):
    texto = texto.lower()           # minúsculas
    texto = re.sub(r'<.*?>', '', texto) # eliminar HTML
    texto = re.sub(r'http\S+', '', texto) # eliminar URLs
    texto = re.sub(r'^a-z\s]', '', texto) # eliminar caracteres especiales
    texto = re.sub(r'\s+', ' ', texto).strip() # espacios extra
    return texto
```

Tokenización

Proceso de dividir el texto en unidades llamadas **tokens**. Los tres tipos más comunes son por palabras, por caracteres y subword (usado en modelos avanzados como BERT).

```
import nltk
nltk.download('punkt', quiet=True)
from nltk.tokenize import word_tokenize

texto = "El modelo funciona bien"
tokens = word_tokenize(texto, language='spanish')
# ['El', 'modelo', 'funciona', 'bien']
```

Stopwords

Palabras sin valor semántico relevante ("el", "la", "de", "y"...) que se eliminan para reducir ruido. Atención: en algunos problemas pueden ser informativas — evaluar caso a caso.

```
from nltk.corpus import stopwords
nltk.download('stopwords', quiet=True)

stop = set(stopwords.words('spanish'))
tokens_filtrados = [t for t in tokens if t not in stop]
```

Normalización: Stemming vs. Lematización

Técnica	Cómo funciona	Precisión	Velocidad	Ejemplo
Stemming	Recorta la palabra a su raíz (heurístico)	Baja	Rápido	"corriendo" -> "corr"
Lematización	Reduce a la forma base real del diccionario	Alta	Más lento	"corriendo" -> "correr"

```
from nltk.stem import SnowballStemmer
import spacy

stemmer = SnowballStemmer('spanish')
print(stemmer.stem('corriendo')) # 'corr'

nlp = spacy.load('es_core_news_sm')
doc = nlp('corriendo')
print([token.lemma_ for token in doc]) # ['correr']
```

10.2. Representaciones de Texto

Los modelos solo entienden números. Una vez limpio y normalizado el texto, se convierte en vectores numéricos mediante alguno de estos métodos:

Bag of Words (BoW)

Cuenta la frecuencia de aparición de cada palabra. Simple pero ignora el contexto y genera vectores muy dispersos.

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = ["el modelo aprende", "el modelo predice", "los datos importan"]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print(X.toarray())
```

TF-IDF

Pondera la frecuencia de una palabra en un documento frente a su frecuencia en el corpus completo: **penaliza las palabras muy comunes** y destaca las palabras clave.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(ngram_range=(1, 2)) # unigramas + bigramas
X = tfidf.fit_transform(corpus)
```

El parámetro `ngram_range=(1,2)` incluye también **bigramas** ("machine learning"), capturando contexto local que BoW pierde.

Embeddings

Representan palabras como **vectores densos** que capturan relaciones semánticas: palabras similares tienen vectores cercanos. Son la base de los modelos modernos de NLP.

```
from sentence_transformers import SentenceTransformer
```

```

model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
embeddings = model.encode(["El modelo aprende", "El algoritmo entrena"])
# embeddings.shape -> (2, 384)

```

Método	Captura contexto	Dimensionalidad	Cuándo usar
Bag of Words	No	Alta (dispersa)	Baseline rápido, datos pequeños
TF-IDF	Parcial (N-gramas)	Alta (dispersa)	Clasificación de texto clásica
Embeddings	Sí	Baja (densa)	Cuando la semántica importa, modelos avanzados

11. Desbalanceo de Datos

11.1. Desbalanceo de Datos: Concepto y Métricas

Un dataset está **desbalanceado** cuando las clases tienen frecuencias muy distintas. El modelo tiende a ignorar la clase minoritaria y predecir siempre la mayoritaria — obteniendo un accuracy aparentemente alto pero siendo inútil en la práctica.

Ejemplo: con 99% de casos normales y 1% de fraude, un modelo que siempre predice "normal" alcanza 99% de accuracy, pero **detecta cero fraudes**.

Detección del desbalanceo

```
# Distribución de clases
print(df['target'].value_counts())
print(df['target'].value_counts(normalize=True) * 100) # en %

import matplotlib.pyplot as plt
df['target'].value_counts().plot(kind='bar')
```

Métricas adecuadas para datos desbalanceados

La **accuracy** no es válida en estos casos. Las métricas relevantes son:

Métrica	Fórmula	Qué mide
Precision	$TP / (TP + FP)$	De todos los positivos predichos, ¿cuántos son realmente positivos?
Recall	$TP / (TP + FN)$	De todos los positivos reales, ¿cuántos detectamos?
F1-Score	$2 * (P * R) / (P + R)$	Balance entre Precision y Recall
ROC-AUC	Área bajo curva ROC	Capacidad global de discriminación entre clases
PR-AUC	Área bajo curva Precision-Recall	Más informativa que ROC-AUC en desbalanceo severo

```
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(f"ROC-AUC: {roc_auc_score(y_test, y_proba):.3f}")
```

¿Qué métrica elegir?

Problema	Métrica principal
Detección de fraude	Recall (minimizar falsos negativos)
Filtro de spam	Precision (minimizar falsos positivos)
Clasificación general	F1-Score

11.2. Técnicas para Conjuntos Desbalanceados

A nivel de datos

```
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import RandomOverSampler, SMOTE

# Undersampling: reduce la clase mayoritaria
rus = RandomUnderSampler(random_state=42)
X_res, y_res = rus.fit_resample(X_train, y_train)

# Oversampling: duplica ejemplos de la clase minoritaria
ros = RandomOverSampler(random_state=42)
X_res, y_res = ros.fit_resample(X_train, y_train)

# SMOTE: genera ejemplos sintéticos interpolando entre vecinos cercanos
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X_train, y_train)
```

Método	Riesgo principal
Undersampling	Pérdida de información de la clase mayoritaria
Oversampling	Sobreajuste al duplicar ejemplos reales
SMOTE	Puede generar ejemplos sintéticos irreales o con ruido

A nivel de algoritmo

Ajuste de pesos — penaliza los errores en la clase minoritaria durante el entrenamiento, sin tocar los datos:

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

model = LogisticRegression(class_weight='balanced')
model = RandomForestClassifier(class_weight='balanced')
```

Umbral de decisión — por defecto los clasificadores usan 0.5 como umbral. Bajarlo aumenta el Recall a costa de más falsos positivos:

```
import numpy as np
from sklearn.metrics import f1_score

y_proba = model.predict_proba(X_test)[:, 1]

thresholds = np.arange(0.1, 0.9, 0.05)
f1_scores = [f1_score(y_test, y_proba >= t) for t in thresholds]
umbral_optimo = thresholds[np.argmax(f1_scores)]

y_pred_ajustado = (y_proba >= umbral_optimo).astype(int)
```